Autonomous Multi Layered AIoT Architecture for Precision Agriculture: Spatial Mapping, Cloud Based Vision, and Reinforcement Learning Driven Irrigation

Tiya Rose
School of Computer Science and Data Science
Vidyashilp University
Bengaluru, India

Vaidhav Adit
School of Computer Science and Data Science
Vidyashilp University
Bengaluru, India

***Abstract*—Efficient resource management in precision agriculture is often hindered by the lack of spatial awareness and adaptive decision logic in conventional irrigation systems. This work presents a multitiered AIoT framework integrating a mobile sensing robot, Arjuna, with a decentralized network of ESP32 nodes and a cloud based intelligence layer. Spatial context is established via Simultaneous Localization and Mapping (SLAM) for global coordinate reporting, while a cloud-hosted YOLOv8 object detection pipeline enables localized crop health monitoring. Decision-making is handled by a Reinforcement Learning (RL) agent utilizing a Deep Q-Network (DQN) trained on an 8-dimensional state vector, incorporating soil moisture, reservoir levels, weather telemetry, and biological growth metrics. To facilitate low-latency cloud communication, edge-side algorithms for spatial matrix transformation and automated map cropping were developed, achieving a 96.83% reduction in transmitted data volume. Benchmark results on the NVIDIA Jetson Nano edge platform demonstrate the system's ability to maintain high-fidelity coordinate mapping and deterministic irrigation actuation at inference under real-time constraints in unstructured environments.**

***Index Terms*—Precision Agriculture, Reinforcement Learning, SLAM, AIoT, YOLOv8, Autonomous Robots, Edge Computing.**

I. Introduction

Precision agriculture faces mounting pressure from water scarcity, climate variability, and food security. Traditional irrigation systems typically operate using static threshold based rules and are unable to dynamically adapt to changing environmental conditions such as soil moisture variability, weather fluctuations, or crop growth stages. As agricultural environments become increasingly heterogeneous, there is a growing need for systems capable of autonomous perception, adaptive decision making, and spatially aware intervention.

The integration of Artificial Intelligence (AI), Internet of Things (IoT), and robotic systems has enabled intelligent farming ecosystems capable of real time monitoring. Existing solutions often operate in isolated layers: IoT lacks autonomous reasoning, mapping systems lack agricultural intelligence, and RL irrigation work often remains simulation bound.

The primary challenge addressed in this work is the absence of a unified architecture that combines spatial mapping, cloud vision intelligence, and adaptive irrigation control into a single closed-loop agricultural system. Existing systems rarely associate crop health or sensor readings with precise global coordinates, which makes localized intervention difficult at scale.

This paper proposes a multi-layered AIoT architecture integrating a mobile robotic platform (Arjuna) for autonomous navigation and spatial mapping, ESP32 based sensing nodes for environmental monitoring, cloud based YOLOv8 object detection for crop maturity analysis, a Deep Q Network based RL agent for irrigation control, and edge optimized coordinate transformation and map compression algorithms.

The contribution of this work lies in the fusion of SLAM driven coordinate awareness with RL based irrigation intelligence. Unlike traditional smart irrigation systems, the proposed framework creates a spatially indexed agricultural intelligence layer capable of associating crop health and irrigation actions with exact physical locations.

II. Critical Review of Related Literature

Prior work in reinforcement learning and robotic mapping motivates the gaps targeted by the proposed architecture, as discussed in the following subsections.

A. PPO-Based Irrigation and Simulation Constraints

Saikai et al. [1] demonstrated the effectiveness of Proximal Policy Optimization (PPO) for irrigation scheduling in maize cultivation. Their work achieved nearly 29% water savings by learning an optimized irrigation policy from simulated environmental interactions. Although the framework demonstrated promising water efficiency, the environment was entirely simulation based and lacked deployment realism.

A major limitation is the assumption of ideal sensor feedback. Real world capacitive moisture sensors exhibit drift and noise; MQTT latency and hardware faults were not considered.

The environment was spatially homogeneous, so the system could not differentiate moisture across field regions.

B. High Dimensional Sensor Feedback and State-Space Complexity

Research presented in [2] explored high dimensional environmental vectors for RL driven irrigation using weather forecasts, layered soil moisture, and evapotranspiration. While decision quality improved, dimensionality increased computational overhead and convergence time, which is problematic on edge devices. The framework remained decoupled from spatial intelligence.

C. SLAM Based Robotic Navigation and the Absence of Agricultural Intelligence

Al-Tawil et al. [3] proposed Enhanced Gmapping (EGM) with PC2LS projection for efficient robotic navigation in unstructured environments, improving mapping precision and reducing computational latency. The work focused on navigation only: the robot could not interpret crop conditions or trigger agricultural actions.

D. Research Gap

The literature treats mapping, crop monitoring, and irrigation intelligence as isolated domains. The proposed architecture addresses this gap through a closed loop AIoT ecosystem integrating SLAM, YOLOv8 based crop monitoring, and RL driven irrigation control.

III. Proposed System Architecture and Methodology

A. System Overview

The proposed framework follows a multilayer AIoT architecture composed of four tightly integrated subsystems: the Robot Execution Layer, the Edge Intelligence Layer, the Cloud Intelligence Layer, and the IoT Irrigation Layer. The complete operational architecture is illustrated in Fig. 2.

The Robot Execution Layer handles physical navigation, environmental sensing, and spatial localization. The Arjuna robot continuously performs SLAM based occupancy mapping while simultaneously estimating pose coordinates using AMCL and multi sensor fusion. During traversal, RGB imagery of crops is captured together with global coordinate metadata represented as (x, y, θ).

The Edge Intelligence Layer operates on the NVIDIA Jetson Nano platform as an intermediate processing node between the robot and cloud infrastructure. This layer performs timestamp synchronization, coordinate formatting, image compression, occupancy matrix optimization, and MQTT payload preparation before forwarding telemetry to the AWS cloud environment.

The Cloud Intelligence Layer consists of five major modules: MQTT broker services, YOLOv8 inference, reinforcement learning inference, environmental weather integration, and centralized database storage. MQTT acts as the communication backbone for asynchronous telemetry exchange between all system components. The YOLOv8 detection model performs tomato ripeness classification with bounding box localization and produces biological growth metrics which are then integrated into the RL state vector.

The RL decision engine receives environmental telemetry, weather conditions, crop maturity information, reservoir status, and historical irrigation behavior to generate adaptive irrigation actions. The resulting irrigation command is transmitted back through MQTT toward ESP32 based BRC nodes responsible for physical irrigation actuation.

The BRC IoT Layer contains capacitive soil moisture sensors, water level monitoring modules, ESP32 communication controllers, and relay controlled irrigation pumps. Once an irrigation command is received, the ESP32 activates the physical irrigation system while continuously publishing updated environmental telemetry back to the cloud layer, thereby forming a fully closed-loop autonomous agricultural intelligence framework.

B. Robot Execution Layer

The Robot Execution Layer forms the physical sensing and navigation subsystem of the architecture. The Arjuna mobile robot integrates ultrasonic sensors for obstacle avoidance, an Inertial Measurement Unit (IMU) for orientation estimation, motor encoders for odometry, an onboard RGB camera for crop monitoring, and NVIDIA Jetson edge hardware for local computation.

The robot performs autonomous traversal while continuously estimating its pose using fused IMU and encoder feedback. Pose estimation is critical for maintaining consistent global coordinate reporting during motion.

C. Arjuna Hardware Platform

The proposed agricultural framework was deployed using the Arjuna autonomous robotic platform. The robot is built on the Robot Operating System (ROS) architecture and integrates multiple sensing, computation, and power management modules required for autonomous agricultural operation. The platform was selected due to its modular hardware design, embedded AI compatibility, and support for autonomous navigation workflows.

The onboard computational layer consists of an NVIDIA Jetson Nano edge AI module coupled with a 64 GB storage system and integrated Wi-Fi communication support. The navigation subsystem incorporates a LiDAR sensor for SLAM based occupancy mapping, BNO055 IMU sensors for orientation estimation, wheel encoder feedback for odometry correction, and ultrasonic sensors for obstacle detection. The robot additionally includes RGB camera modules for crop monitoring and image acquisition during navigation.

The mobility platform supports both rubber wheel and mecanum wheel configurations, enabling stable traversal across structured indoor agricultural environments. Smart servo motors connected through a dedicated expansion board provide motion control and path execution. The robot chassis additionally integrates a Smart Robot Manager Board capable of voltage monitoring, current regulation, thermal monitoring,

automated power shutdown, and overcharge protection. These hardware level safeguards improve operational stability during long duration agricultural deployment.

Power management is handled through a Li-Ion battery pack integrated with a Battery Management System (BMS). The BMS continuously regulates overcurrent conditions, battery balancing, temperature monitoring, and short circuit protection to ensure reliable operation of edge computation and sensing components.

D. IoT Sensing Layer

The IoT layer consists of distributed ESP32 based BRC nodes responsible for environmental monitoring and irrigation actuation. Each node incorporates capacitive soil moisture sensors, water level sensors, relay controlled water pumps, and MQTT communication modules. The use of asynchronous MQTT publishing ensures lightweight communication and low transmission overhead. Sensor readings are periodically published to the cloud layer for decision making.

E. Spatial Mapping and Coordinate Reporting

A major contribution of the proposed architecture is the integration of real world coordinate mapping into agricultural monitoring.

Arjuna utilizes Simultaneous Localization and Mapping (SLAM) to generate a 2D occupancy grid representation of the environment. Adaptive Monte Carlo Localization (AMCL) is used for estimating the robot pose represented as:

$$(x, y, \theta) \tag{1}$$

where x and y represent global coordinates while θ represents robot orientation.

Whenever a crop image is captured, the corresponding spatial coordinates are attached to the image payload and transmitted to the cloud. This creates a spatially indexed crop intelligence database capable of associating irrigation decisions with exact physical locations.

F. Coordinate Transformation Methodology

The SLAM occupancy grid is transformed into a mathematically consistent coordinate framework using affine transformation matrices.

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} res & 0 & ox \\ 0 & res & oy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{2}$$

where res is map resolution, ox and oy are map origin coordinates, (u, v) are pixel coordinates, and (x, y) are real world coordinates.

The transformation matrix enables pixel level coordinate consistency throughout the environment.

TABLE I: RL State Vector Description

Parameter	Description
Soil Moisture	Current soil water content
Water Tank Level	Remaining reservoir capacity
Growth Score	Biological crop growth metric
Normalized Day	Crop lifecycle progression
Temperature	Environmental temperature
Humidity	Atmospheric humidity level
Rain Probability	Predicted rainfall telemetry
Previous Action	Previous irrigation decision

G. Vision Intelligence Layer

The cloud intelligence layer utilizes a YOLOv8 based object detection pipeline for crop ripeness recognition. The YOLOv8 model classifies tomatoes into three categories: `not_ripe`, `half_ripe`, and `ripe`. The anchor free detection head design in YOLOv8 improves localization accuracy while maintaining efficient inference. Upon receiving an image through MQTT, the cloud subscriber performs inference and stores the class scores and bounding boxes along with the corresponding coordinates.

H. YOLOv8 Dataset and Training Configuration

The tomato ripeness recognition model was trained on the Tomato Ripeness dataset from Roboflow Universe [4], using axis-aligned bounding boxes in YOLO TXT format, varied lighting and viewing angles, and mixed illumination. Data augmentation included horizontal flipping, random scaling, and brightness variation; data were split for training and validation to assess detection consistency at inference time.

I. Reinforcement Learning Framework

The irrigation decision engine was modeled as a Markov Decision Process (MDP) where each timestep corresponds to one operational cycle within the agricultural environment. The reinforcement learning subsystem continuously evaluates environmental conditions and determines whether irrigation should be activated based on long term crop growth optimization.

The RL environment models dynamic agricultural behavior including evaporation, soil moisture depletion, reservoir consumption, crop growth progression, and changing weather conditions. Environmental randomness was introduced using stochastic perturbation of weather variables to improve policy generalization and prevent overfitting during training.

The agent observes an 8 dimensional normalized state vector represented as:

$$s \in \mathbb{R}^8 \tag{3}$$

The state representation combines environmental telemetry, crop intelligence metrics, and historical irrigation behavior. Table I summarizes the parameters included within the RL state vector.

The action space was designed as a binary irrigation control system directly compatible with ESP32 relay actuation. Action 0 corresponds to no irrigation while Action 1 activates the irrigation pump.

A Deep Q-Network (DQN) architecture was selected due to its stability under continuous environmental state transitions. The neural network consists of an 8 neuron input layer, two hidden dense layers containing 128 neurons each, and a final output layer representing Q values for the two irrigation actions. Experience replay memory and target network synchronization were incorporated to improve convergence stability during training.

The reward structure was designed to balance sustainable water utilization against healthy crop growth progression. The reward function is represented as:

$$R = -(5.0 \cdot |G_{actual} - G_{expected}|) - P_{water} - P_{empty} \quad (4)$$

where G_{actual} denotes observed plant growth, $G_{expected}$ denotes expected growth progression, P_{water} penalizes excessive irrigation, and P_{empty} penalizes reservoir depletion conditions.

Additional reward shaping constraints were incorporated to discourage irrigation during saturated soil conditions and to penalize irrational pump activation when reservoir levels were critically low. A terminal harvest reward was additionally introduced near the end of each cultivation episode to encourage long horizon irrigation planning behavior.

Among Q-Learning, SARSA, PPO, REINFORCE, Actor-Critic, and DQN evaluated in the same environment, discrete tabular value methods did not scale to the continuous eight-dimensional state, and several policy-gradient variants exhibited higher training variance in our runs; DQN provided the most stable irrigation policy in this setting and was deployed.

J. RL Training Configuration

Training was performed using PyTorch with the Adam optimizer under a learning rate of 0.001 and discount factor (γ) of 0.99. The replay batch size was fixed at 64 while target network synchronization was performed every 10 episodes.

Epsilon greedy exploration was initialized at 1.0 and progressively decayed during training until reaching a minimum threshold of 0.05. The complete training cycle consisted of 500 episodes totaling approximately 45,000 simulated operational days in the agricultural environment model (about 90 simulated days per episode).

All experiments were executed using Python 3.12 with NumPy based environmental simulation and MQTT integrated telemetry communication. Fig. 1 summarizes reward convergence across the RL algorithms compared during development (final figure as produced from training logs).

K. System Architecture Diagram

Fig. 2 illustrates the complete operational architecture of the proposed AIoT framework integrating robotic navigation, edge preprocessing, cloud intelligence, and IoT based irrigation actuation.

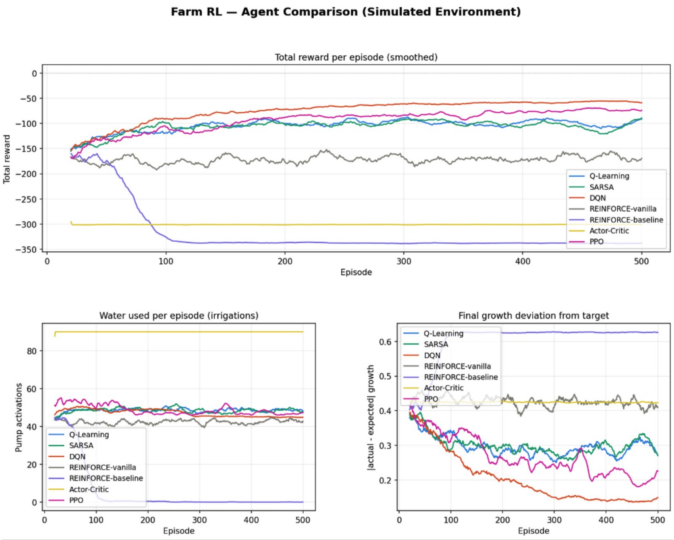


Fig. 1: RL training reward convergence comparison between DQN, PPO, SARSA, Q-Learning, REINFORCE, and Actor-Critic over 500 training episodes.

L. Operational Data Flow

The operational workflow shown in Fig. 2 begins within the Robot Execution Layer where the Arjuna platform performs SLAM based occupancy mapping and AMCL driven localization using LiDAR, IMU, wheel encoder, and ultrasonic sensor fusion. During navigation, RGB crop imagery is captured together with associated spatial coordinates represented as (x, y, θ).

The collected telemetry is forwarded toward the Edge Intelligence Layer operating on NVIDIA Jetson hardware. This layer performs image compression, timestamp synchronization, occupancy matrix optimization, coordinate transformation, and MQTT packet preparation before cloud transmission.

Within the Cloud Intelligence Layer shown in Fig. 2, the MQTT broker manages asynchronous communication between robotic agents, sensing nodes, and cloud inference services. The YOLOv8 detector performs tomato ripeness recognition with bounding box outputs and generates crop intelligence metrics including tomato count and green score estimation.

Weather telemetry from OpenWeather, including temperature, humidity, wind speed, solar radiation, evaporation estimates, and rainfall probability, is integrated into the reinforcement learning state representation. The RL engine subsequently evaluates the aggregated environmental state vector and generates irrigation decisions using the trained DQN policy.

Finally, irrigation commands are transmitted back through MQTT toward ESP32 based BRC nodes located within the IoT irrigation layer. Relay controlled pumps execute irrigation actions while updated soil moisture and reservoir telemetry are continuously republished back to the cloud, thereby establishing a fully closed loop agricultural intelligence framework.

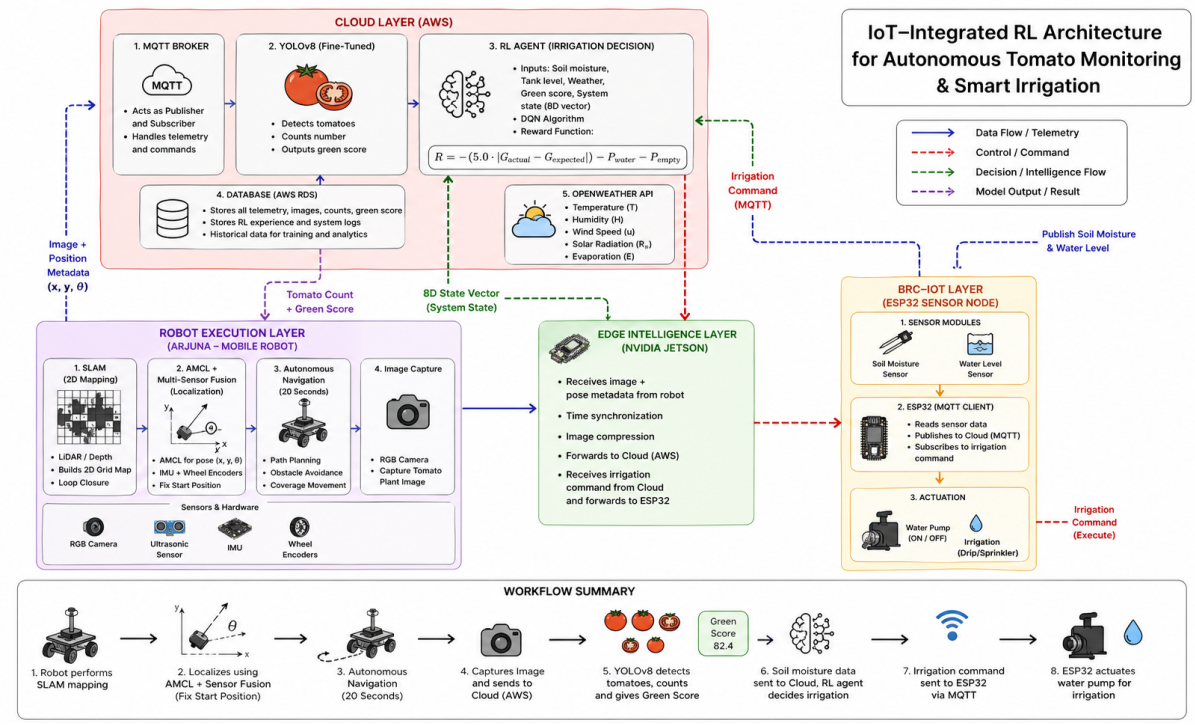


Fig. 2: Complete AIoT architecture integrating SLAM based robotic mapping, edge intelligence, AWS cloud vision inference, reinforcement learning based irrigation control, and ESP32 driven irrigation actuation.

IV. Tomato Ripeness Detection Using YOLOv8

This section details the fine-tuning of a YOLOv8 object detection model for tomato ripeness classification, which serves as the cloud-tier vision component of the proposed AIoT framework. The model provides per-fruit ripeness labels and confidence scores that are spatially indexed against Arjuna's SLAM coordinates and consumed by the RL irrigation agent.

A. Dataset

The training data was sourced from the Tomato Ripeness dataset on Roboflow Universe [4], comprising 1,362 annotated images across three splits: 1,230 training, 85 validation, and 47 test images. All annotations are in YOLOv8 TXT format with normalized bounding box coordinates. Three ripeness classes are defined:

- **`not_ripe`** — Fully green, unripe tomatoes. Most visually distinct class.
- **`half_ripe`** — Transitional tomatoes with mixed green, orange, and yellow tones. Most ambiguous class.
- **`ripe`** — Fully red, mature tomatoes ready for harvest. Most commercially relevant class.

B. Model Selection and Fine-Tuning

YOLOv8m (medium variant) was selected as the detection backbone, offering 25.8 million parameters and 78.7 GFLOPs, providing an optimal trade-off between accuracy and efficiency for the NVIDIA Tesla T4 cloud-tier GPU (14.9 GB VRAM). The model was initialized with COCO-pretrained weights [5] encoding generalizable visual features across 80 object categories including produce.

Fine-tuning was performed on a Kaggle-hosted T4 GPU using the Ultralytics library. Training ran for a maximum of 50 epochs with early stopping at patience 15. The optimizer was AdamW with learning rate 1.429×10^{-3}, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and default weight decay per Ultralytics configuration. Automatic Mixed Precision (AMP) reduced GPU memory by approximately 40%. Batch size was 8 with input resolution 640×640 pixels.

Augmentations followed the Ultralytics recipe: mosaic, horizontal flip, HSV jitter (hue, saturation, value), random scale, translation, and rotation, emphasizing HSV because color drives ripeness separation.

C. Results and Analysis

Training converged at epoch 27 with early stopping at epoch 35. Close tracking of train and validation loss curves confirmed generalization without overfitting. Quantitative results are reported in Table II and Table III.

The val-to-test gap of approximately 2–3% indicates stable generalization. `ripe` and `not_ripe` show strong AP; `half_ripe` is harder, consistent with [6]. Fig. 3 shows the training dashboard: losses, mAP curves, and per-class AP on the test split (best epoch 27 marked).

D. Integration with the AIoT Framework

The fine-tuned model runs as a cloud MQTT subscriber: it outputs boxes, classes, and scores tied to Arjuna SLAM-

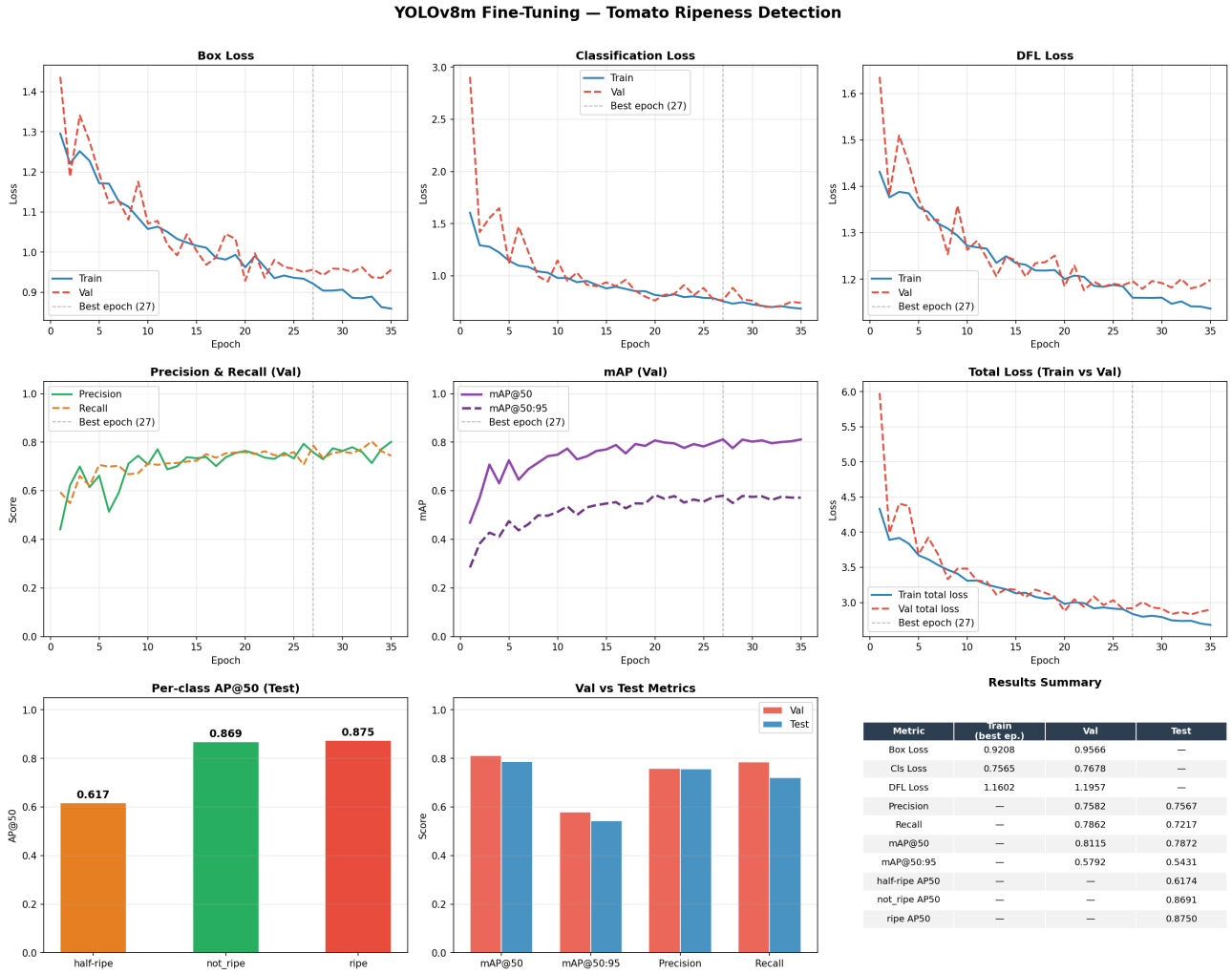


Fig. 3: YOLOv8m fine-tuning over 35 epochs: train versus validation box, classification, and DFL loss; precision, recall, and mAP; per-class AP@50 on the test set; val versus test summary. Vertical dashed line: epoch 27.

TABLE II: YOLOv8m Fine-Tuning Results — Tomato Ripeness

Metric	Train	Val	Test
Box Loss	0.9208	0.9566	—
Cls Loss	0.7565	0.7678	—
DFL Loss	1.1602	1.1957	—
Precision	—	0.7582	0.7567
Recall	—	0.7862	0.7217
mAP@50	—	0.8115	0.7872
mAP@50:95	—	0.5792	0.5431

TABLE III: Per-Class AP@50 on Test Set

Class	AP@50	Instances
`not_ripe`	0.8691	157
`half_ripe`	0.6174	109
`ripe`	0.8750	164
All (mean)	**0.7872**	**430**

AMCL (x, y) for a spatial ripeness map that feeds the RL state and the operator dashboard.

V. Optimization of Edge Layer Algorithmic Routines

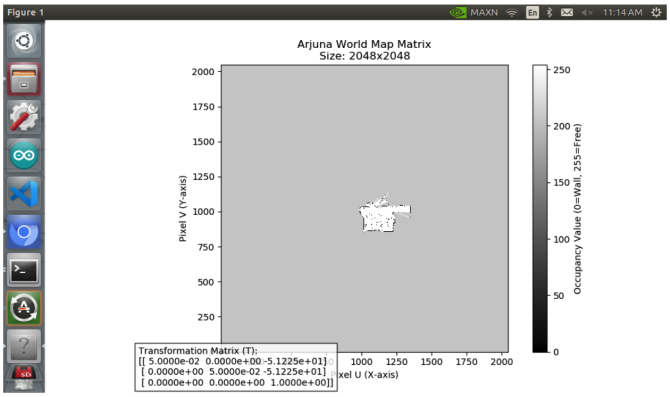
The edge layer is responsible for reducing communication latency and minimizing computational overhead before cloud transmission.

A. World Matrix Generation Algorithm

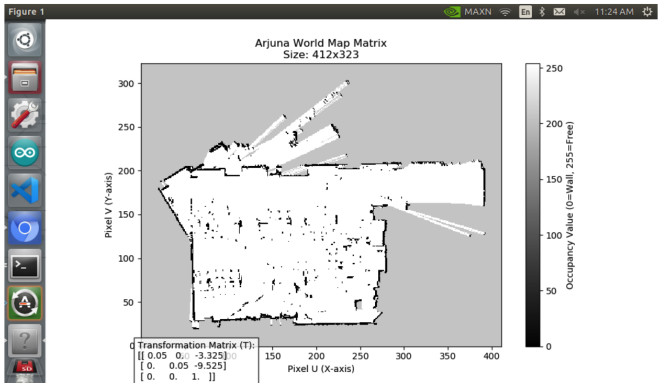
The World Matrix Generation algorithm converts occupancy grid maps into spatially aware coordinate systems. YAML metadata is parsed to extract map resolution and origin coordinates, after which a 3×3 affine transformation matrix is constructed. NumPy ndarrays are used for vectorized matrix operations, enabling efficient batch transforms on NVIDIA Jetson hardware.

B. Dynamic ROI Extraction and Map Compression

The Automated Map Cropping algorithm identifies the active region of interest within the occupancy grid. Boolean indexing marks occupied cells; matrix slicing removes large



(a) Raw occupancy grid from SLAM.



(b) Cropped ROI transmitted to cloud.

Fig. 4: Occupancy grid before and after ROI extraction for edge transmission.

unexplored margins. Coordinate offsets are updated so cropped pixels still map to the same world frame before MQTT upload.

Fig. 4 ties this procedure to the SLAM maps: Fig. 4a shows the dense occupancy grid after mapping, including unexplored padding; Fig. 4b shows the ROI-limited grid actually transmitted after removing that padding. Visually, the second panel matches the compressed payload that yields the reported 96.83% size reduction relative to the full matrix.

C. Edge Versus Cloud Communication Analysis

The proposed edge optimization pipeline significantly reduced the payload transmitted to the cloud infrastructure. ROI extraction before transmission cut payload by 96.83%, improving MQTT efficiency and cloud responsiveness on AWS.

VI. Experimental Performance Evaluation and Discussion

This section evaluates the computational efficiency and real time suitability of the proposed architecture.

A. Experimental Setup

Profiling used the Arjuna stack with NVIDIA Jetson Nano edge hardware in an indoor mapped environment, with ESP32

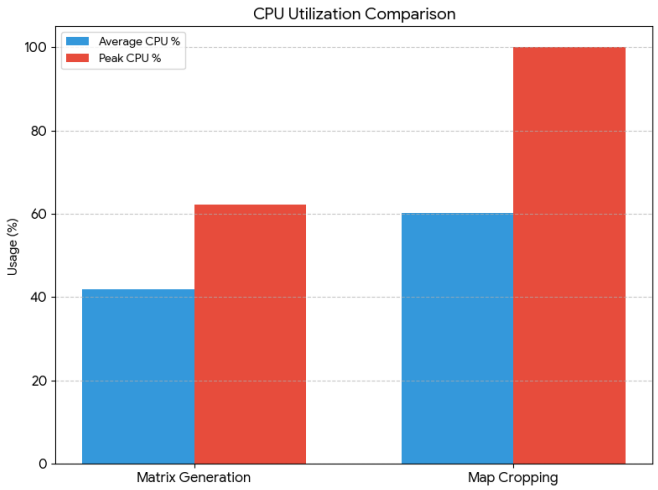


Fig. 5: CPU utilization comparison between matrix transformation and ROI extraction routines.

sensing nodes, capacitive soil moisture sensors, MQTT, and AWS subscribers. CPU, memory, latency, and power were recorded over repeated runs of the matrix generation and ROI routines.

B. CPU Utilization Characteristics

CPU utilization analysis was performed for the Matrix Generation and ROI Cropping algorithms. As illustrated in Fig. 5, the Matrix Generation routine maintained an average CPU utilization of 41.83%, whereas ROI extraction exhibited higher temporary peaks due to intensive matrix slicing operations.

The trends shown in Fig. 5 demonstrate that both algorithms remain computationally feasible for deployment on embedded AI hardware despite temporary processing spikes during ROI extraction.

C. Memory Footprint Assessment

Memory profiling was conducted to evaluate RAM consumption under continuous operation. Fig. 6 presents the memory utilization characteristics of both spatial optimization routines.

As shown in Fig. 6, the Matrix Generation algorithm reached a peak memory usage of 87 MB, while ROI extraction operated within 62 MB. The relatively low memory footprint demonstrates suitability for resource constrained edge environments such as NVIDIA Jetson platforms.

D. Latency Profiling

Latency is a critical metric in autonomous robotic systems because delayed coordinate updates directly affect navigation consistency and irrigation responsiveness. The execution latency comparison between the two edge algorithms is illustrated in Fig. 7.

The results shown in Fig. 7 indicate that the ROI extraction algorithm completed execution in 252.05 ms, while full matrix generation required approximately 1617.83 ms. This

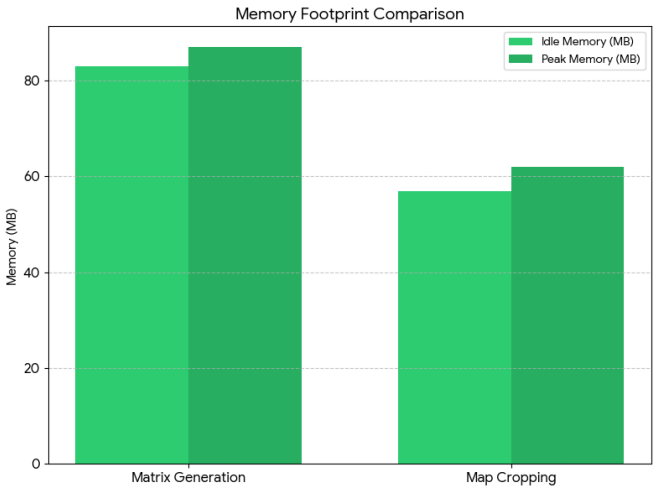


Fig. 6: Memory utilization analysis for edge-layer spatial optimization algorithms.

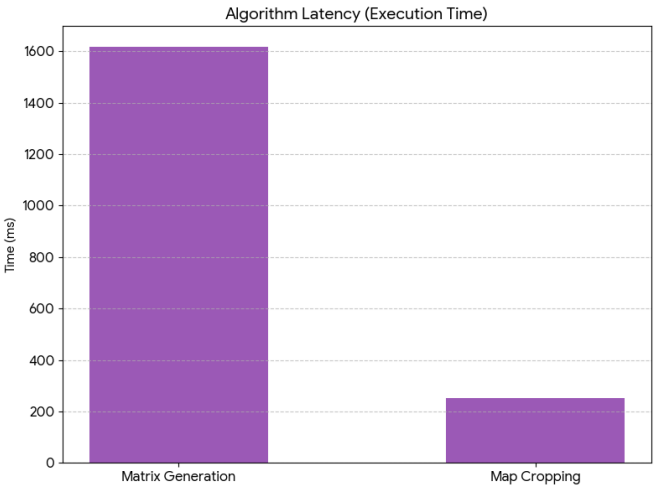


Fig. 7: Latency comparison between occupancy grid transformation and ROI extraction (time in ms).

substantial reduction in execution time enables faster cloud synchronization and improved real time responsiveness.

E. Power Consumption Metrics

Power profiling was performed to evaluate energy efficiency during edge computation. Fig. 8 illustrates the estimated power consumption characteristics during high intensity processing operations.

As observed in Fig. 8, peak power consumption reached approximately 30.14 W during ROI extraction and 20.91 W during matrix transformation. Although ROI extraction briefly consumed higher power, its significantly shorter execution duration resulted in lower overall computational overhead.

F. Algorithmic Scalability

The proposed edge optimization framework demonstrated strong scalability characteristics for large scale agricultural environments. The ROI extraction algorithm achieved a 96.83% payload reduction while processing over 16 million pixels per second. The framework additionally reduced MQTT transmission latency while preserving coordinate consistency throughout the environment.

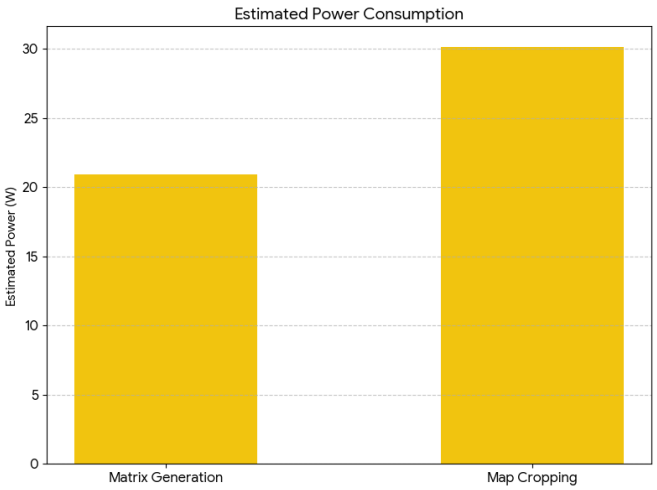


Fig. 8: Estimated power consumption during high intensity edge processing operations.

The computational complexity of the algorithm remains approximately $O(n)$ due to efficient matrix traversal and boolean indexing.

REFERENCES

[1] N. Saikai et al., "Optimizing Water Use in Maize Irrigation with Reinforcement Learning," *Mathematics*, vol. 13, no. 4, p. 595, 2025.
[2] N. Saikai, A. Peake, and K. Chenu, "Deep Reinforcement Learning for Irrigation Scheduling Using High-Dimensional Sensor Feedback," *PLOS Water*, vol. 3, no. 1, 2024.
[3] M. Al-Tawil et al., "Mobile Robot Navigation with Enhanced 2D Mapping and Multi-Sensor Fusion," *Sensors*, vol. 25, no. 8, p. 2408, 2025.
[4] "Tomato Ripeness Dataset," Roboflow Universe, [Online]. Available: https://universe.roboflow.com/dataset-w8sqm/tomato-ripeness-jionb
[5] G. Jocher et al., "Ultralytics YOLOv8," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023.
[6] M. Nahiduzzaman et al., "Deep learning-based real-time detection and classification of tomato ripeness stages using YOLOv8 on Raspberry Pi," *Engineering Research Express*, vol. 7, p. 015219, 2025.